

Построение ДКА по регулярному выражению алгоритмом Глушкова

Проект по курсу “Теория и реализация языков программирования”

17 мая 2026 г.

1 Постановка задачи

В работе рассматривается учебная реализация преобразования регулярного выражения в детерминированный конечный автомат. На вход программе подается строка, задающая регулярное выражение в выбранном синтаксисе. На выходе формируется LaTeX/TikZ-код диаграммы ДКА. Между этими двумя шагами программа строит абстрактное синтаксическое дерево, линейаризует литералы, вычисляет атрибуты Глушкова, строит позиционный автомат и детерминизирует его.

Определение 1. Пусть дано регулярное выражение r над конечным алфавитом Σ . Требуется построить детерминированный конечный автомат A_D такой, что $L(A_D) = L(r)$, и вывести код его диаграммы в формате LaTeX/TikZ.

Проект написан на Haskell и собирается через Cabal. Синтаксический анализ выполнен с помощью библиотеки Parsec. Реализация не претендует на совместимость с PCRE или POSIX regular expressions: входной язык специально оставлен небольшим и формальным, чтобы явно показать этапы алгоритма.

2 Входной язык и нормализация

Пользователь вводит выражение в concrete syntax. Приоритет операций принят обычный: постфиксные операции связывают сильнее конкатенации, а конкатенация сильнее альтернативы.

```
regex ::= alt
alt   ::= concat ("|" concat)*
concat ::= postfix+
postfix ::= atom ("*" | "+" | "?")*
atom   ::= literal | escaped | "(" regex ")" | "ε" | "∅"
```

Пробелы между токенами игнорируются и не входят в алфавит выражения. Обратная косая черта экранирует служебные символы. Например, `\*` означает литерал `*`, а не операцию звезды Клинни.

Операции `+` и `?` являются синтаксическим сахаром. На этапе разбора они раскрываются в базовые операции:

$$r^+ \equiv rr^*, \quad r? \equiv \varepsilon \mid r.$$

После такой нормализации последующие алгоритмы работают уже с ядром регулярных выражений, а не с полным пользовательским синтаксисом.

3 Ядро регулярных выражений

Внутреннее представление использует небольшое ядро:

$$R ::= \emptyset \mid \varepsilon \mid a \mid R_1 R_2 \mid (R_1 + R_2) \mid R^*.$$

Здесь $a \in \Sigma$, а знак $+$ в формуле обозначает альтернативу. Во входной строке для альтернативы используется символ $|$, а в коде ей соответствует конструктор `Alt`.

AST выражения задается следующими вариантами:

- `EmptySet` — пустой язык $\emptyset$;
- `Epsilon` — язык, состоящий только из пустого слова;
- `Literal Char` — один символ алфавита;
- `Concat [Regex]` — конкатенация подвыражений;
- `Alt [Regex]` — альтернатива подвыражений;
- `Star Regex` — итерация Клини.

Замечание 1. `Concat` и `Alt` хранятся списками, потому что парсеру удобно собирать цепочки вроде `abcd` или `a/b/c` без лишних вложенных узлов. Семантически эти списки обрабатываются сверткой по бинарным правилам: `abc` рассматривается как $((ab)c)$, а `a + b + c` как $((a + b) + c)$. Это важно для корректного вычисления nullable-подвыражений и отношения `follow`.

4 Линеаризация

Определение 2. Пусть $\text{Pos}(r) = \{1, \dots, n\}$ — множество позиций литералов в выражении r . Линеаризация сопоставляет каждому вхождению литерала пару (p, a) , где p — уникальный номер позиции, а a — исходный символ.

Например:

$$aa \mapsto a_1 a_2, \quad a(a \mid a) \mapsto a_1(a_2 \mid a_3).$$

Линеаризация не является косметическим переименованием. В выражении `aa` две буквы имеют одинаковый символ, но разные роли: после первой позиции должна идти вторая, а после второй выражение уже может закончиться. Если различать только символы, отношение следования невозможно восстановить корректно.

В реализации исходный тип `Regex` не изменяется. Для алгоритма строится отдельное позиционированное дерево `PositionedRegex`. Таблица `symbolTable` хранит соответствие `Position -> Char` и позволяет на поздних этапах вернуться от номеров позиций к символам входного алфавита.

5 Атрибуты Глушкова

Для позиционированного выражения вычисляются четыре атрибута.

Определение 3. Для регулярного выражения r :

- $\text{nullable}(r)$ истинно тогда и только тогда, когда $\varepsilon \in L(r)$;
- $\text{First}(r)$ — множество позиций, которые могут быть первыми в словах языка $L(r)$;
- $\text{Last}(r)$ — множество позиций, которые могут быть последними;
- $\text{Follow}_r(i)$ — множество позиций, которые могут идти сразу после позиции i в некотором слове языка $L(r)$.

Для пустого языка:

$$\text{nullable}(\emptyset) = \text{false}, \quad \text{First}(\emptyset) = \text{Last}(\emptyset) = \emptyset, \quad \text{Follow} = \emptyset.$$

Для пустого слова:

$$\text{nullable}(\varepsilon) = \text{true}, \quad \text{First}(\varepsilon) = \text{Last}(\varepsilon) = \emptyset, \quad \text{Follow} = \emptyset.$$

Для литерала a_p в позиции p :

$$\text{nullable}(a_p) = \text{false}, \quad \text{First}(a_p) = \text{Last}(a_p) = \{p\}, \quad \text{Follow} = \emptyset.$$

Для альтернативы:

$$\begin{aligned} \text{nullable}(r + s) &= \text{nullable}(r) \vee \text{nullable}(s), \\ \text{First}(r + s) &= \text{First}(r) \cup \text{First}(s), \\ \text{Last}(r + s) &= \text{Last}(r) \cup \text{Last}(s), \\ \text{Follow}(r + s) &= \text{Follow}(r) \cup \text{Follow}(s). \end{aligned}$$

Для конкатенации:

$$\begin{aligned} \text{nullable}(rs) &= \text{nullable}(r) \wedge \text{nullable}(s). \\ \text{First}(rs) &= \begin{cases} \text{First}(r) \cup \text{First}(s), & \text{nullable}(r), \\ \text{First}(r), & \text{иначе.} \end{cases} \\ \text{Last}(rs) &= \begin{cases} \text{Last}(r) \cup \text{Last}(s), & \text{nullable}(s), \\ \text{Last}(s), & \text{иначе.} \end{cases} \\ \text{Follow}(rs) &= \text{Follow}(r) \cup \text{Follow}(s) \cup \{i \mapsto \text{First}(s) \mid i \in \text{Last}(r)\}. \end{aligned}$$

Для звезды Клини:

$$\begin{aligned} \text{nullable}(r^*) &= \text{true}, \quad \text{First}(r^*) = \text{First}(r), \quad \text{Last}(r^*) = \text{Last}(r). \\ \text{Follow}(r^*) &= \text{Follow}(r) \cup \{i \mapsto \text{First}(r) \mid i \in \text{Last}(r)\}. \end{aligned}$$

Замечание 2. В программе `follow` хранится как `Map Position (Set Position)`. Это разреженное представление бинарного отношения следования: если для позиции нет продолжений, отдельная запись в таблице не нужна.

6 Позиционный автомат Глушкова

По вычисленным атрибутам строится позиционный автомат

$$A_G = (Q, \Sigma, \delta, 0, F).$$

Если после линеаризации выражение имеет n позиций, то

$$Q = \{0\} \cup \text{Pos}(r).$$

Состояние 0 является начальным и не соответствует никакому символу. Каждая позиция $p \geq 1$ имеет метку $\text{sym}(p)$ из таблицы символов.

Переходы задаются двумя правилами:

- если $p \in \text{First}(r)$, то $\delta(0, \text{sym}(p))$ содержит p ;
- если $j \in \text{Follow}_r(i)$, то $\delta(i, \text{sym}(j))$ содержит j .

Множество принимающих состояний:

$$F = \text{Last}(r) \cup \begin{cases} \{0\}, & \text{nullable}(r), \\ \emptyset, & \text{иначе.} \end{cases}$$

Автомат Глушкова не содержит ε -переходов. При этом после замены позиций обычными символами он может стать недетерминированным. Например, для выражения $a|a$ из состояния 0 по символу a есть два перехода: в позицию 1 и в позицию 2.

7 Детерминизация

Из позиционного автомата строится ДКА

$$A_D = (Q_D, \Sigma, \delta_D, q_0, F_D)$$

методом `reachable subset construction`. Начальное состояние ДКА соответствует подмножеству

$$q_0 = \{0\}.$$

Выбор именно $\{0\}$ отражает состояние позиционного автомата до чтения первого символа входного слова.

Для $S \subseteq Q$ и символа $c \in \Sigma$:

$$\text{move}(S, c) = \bigcup_{q \in S} \delta(q, c).$$

Если $\text{move}(S, c)$ непусто и ранее не встречалось, для него создается новое состояние ДКА. Таким образом строятся только достижимые подмножества, а не весь булеан множества Q .

Принимающие состояния ДКА определяются правилом:

$$S \in F_D \iff S \cap F \neq \emptyset.$$

В реализации ДКА является частичным: отсутствующий переход по очередному символу означает немедленное отклонение слова. Это не ошибка модели, а практическое представление автомата для проверки принадлежности. При необходимости его можно тотализировать добавлением отдельного `sink`-состояния.

8 Генерация LaTeX/TikZ

Генератор выдает `standalone`-документ LaTeX. Диаграмма строится внутри окружения `tikzpicture` с использованием стандартных стилей библиотеки `automata`: `state`, `initial` и `accepting`.

Каждое состояние ДКА q_i выводится как вершина. Перед диаграммой печатаются комментарии вида:

```
% q0 = {0}
% q1 = {1, 3}
```

Эти строки показывают, как состояния ДКА связаны с подмножествами состояний позиционного автомата, то есть делают результат subset construction видимым в сгенерированном файле.

Самопетли и обычные переходы генерируются автоматически. Если несколько символов ведут из одного состояния в одно и то же состояние, они группируются в одну подпись ребра. Специальные символы LaTeX в метках экранируются, поэтому литералы вроде `*`, `_` или `%` можно безопасно выводить в диаграмме.

9 Структура реализации

Модуль	Назначение
<code>Regex.Syntax</code>	AST и pretty-print ядра регулярных выражений.
<code>Regex.Parser</code>	Concrete syntax, приоритеты операций и раскрытие сахара.
<code>Regex.Linearize</code>	Позиционирование литералов и таблица <code>Position -> Char</code> .
<code>Automata.Glushkov</code>	Атрибуты <code>nullable/first/last/follow</code> и позиционный автомат.
<code>Automata.DFA</code>	Reachable subset construction, DFA и проверка слова.
<code>Render.Tikz</code> <code>app/Main.hs</code>	Генерация standalone LaTeX/TikZ-документа. CLI-режимы для AST, атрибутов, автоматов, match и TikZ.
<code>test/Main.hs</code>	Набор unit/smoke-проверок для основных этапов.

Такое разделение сохраняет связь между теорией и кодом: каждый математический этап вынесен в отдельный модуль и может проверяться независимо.

10 Верификация и тесты

Текущая версия содержит 69 unit/smoke проверок. Они покрывают:

- парсер, приоритеты операций и скобки;
- раскрытие синтаксического сахара `+` и `?`;
- экранирование служебных символов;
- линеаризацию повторяющихся символов;
- вычисление `nullable`, `first`, `last` и `follow`;
- NFA-переходы для ε , $\emptyset$, `a`, `ab`, `a*` и `a|a`;
- acceptance-тесты для DFA;
- наличие ключевых фрагментов в сгенерированном TikZ.

Дополнительно сборка LaTeX проверяет, что генерируемый standalone-файл можно передать в pdf_latex без ручного исправления.

11 Сложность

Пусть n — число позиций, то есть число литеральных вхождений в регулярном выражении. Позиционный автомат Глушкова имеет $n + 1$ состояний. Отношение `follow`

в худшем случае может связывать квадратично много пар позиций, поэтому число переходов позиционного автомата ограничено величиной

$$O(n^2).$$

Детерминизация в худшем случае экспоненциальна:

$$O(2^n)$$

состояний ДКА. На практике реализация строит только достижимые подмножества, поэтому для многих выражений фактический автомат оказывается существенно меньше полного множества подмножеств.

12 Пример

Рассмотрим выражение:

$(a|b)^*abb$

Линеаризация дает позиции:

1 = 'a'
2 = 'b'
3 = 'a'
4 = 'b'
5 = 'b'

Атрибуты для всего выражения:

first = {1, 2, 3}
last = {5}

follow:

1 -> {1, 2, 3}
2 -> {1, 2, 3}
3 -> {4}
4 -> {5}

После детерминизации получаются следующие состояния ДКА:

q0 = {0}
q1 = {1, 3}
q2 = {2}
q3 = {2, 4}
q4 = {2, 5}

Состояние q_4 является принимающим, потому что содержит позицию 5 из $\text{Last}(r)$.
Короткие проверки:

aabb -> accept
aba -> reject

13 Сборка и воспроизведение

Основные команды для проверки проекта:

```
cabal build
cabal test
cabal run -v0 glushkov-algo -- --match "(a|b)*abb" "aabb"
cabal run -v0 glushkov-algo -- --tikz "(a|b)*abb" > /tmp/dfa.tex
pdflatex -interaction=nonstopmode -halt-on-error -output-directory /tmp /tmp/dfa.tex
```

Диагностические режимы `--ast`, `--attrs`, `--nfa` и `--dfa` позволяют посмотреть промежуточные структуры без изменения основного алгоритма.

14 Заключение

В проекте реализован полный основной путь: от строки с регулярным выражением до ДКА и LaTeX/TikZ-кода диаграммы. Алгоритмическая часть разделена на этапы, соответствующие построению Глушкова: линейаризация, атрибуты, позиционный автомат и детерминизация. Минимизация ДКА не выполняется, поскольку она не требуется постановкой задачи и является отдельным алгоритмом.

15 Используемые материалы

- Материалы курса “Теория и реализация языков программирования”, репозиторий <https://github.com/rbtsv/TRPL>.
- Классическая теория регулярных языков и конечных автоматов: построение автомата позиций, или алгоритм Глушкова.
- Документация Parsec.
- Документация Cabal.
- Документация TikZ, библиотека automata.